Carrera: Enabling High-Performance eBPF-based Sketches in Network Measurement

Xiang Chen[1], Xin Yao[1], Jiayu Li[1], Longlong Zhu[1], Linying Zheng[1], Hongyan Liu[2], Jianshan Zhang[3], Dong Zhang[2], Xuan Liu[4], Qun Huang[5], Haifeng Zhou[1], Chunming Wu[1]
[1]Zhejiang University [2]Fuzhou University [3]Minjiang University [4]Yangzhou University [5]Peking University

2025 IEEE 33rd International Conference on Network Protocols (ICNP) | 979-8-3315-0376-5/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICNP65844.2025.11192430

***Abstract*—To achieve dynamic network measurement, trends build sketches on eBPF to avoid service interruptions. However, existing eBPF-based sketches suffer from high CPU consumption, leading to poor throughput and high latency and making them hard to measure high-speed traffic. Optimizing their performance requires users to refactor codes based on each sketch's characteristics on eBPF, which is highly complex and time-consuming.**

In this paper, we argue that users should write sketches without concerning low-level eBPF performance optimizations, with the deployment automatically activating cross-sketch performance optimizations. We present Carrera, a library that offers domain-specific optimizations for eBPF-based sketches. Our contributions are (1) systematically analyzing the performance bottlenecks of eBPF-based sketches through microbenchmarks, (2) identifying practical optimizations, including hardware offloading, SIMD-accelerated hashing, traffic-aware flow index caching, prefetched randomization, and active data collection, to address the identified bottlenecks in eBPF-based sketches, (3) evaluating these optimizations with state-of-the-art sketches and demonstrating that Carrera improves throughput by up to 65% and reduces latency by up to 93% via testbed experiments.

I. Introduction

In recent years, modern production networks have evolved into multi-tenant networks, in which virtualization plays an important role [1, 2, 3]. In detail, each server simultaneously runs tens of virtual machines or containers, which execute multiple tenant services (e.g., video encoding [4] and parallel compilation [5]). As such, traffic measurement has been considered as a fundamental building block of virtualization. It can capture real-time statistics of inter-VM or inter-container traffic in service of various tasks, such as heavy hitter detection [6, 7], per-flow frequency estimation [8], cardinality estimation [9, 10], and traffic change detection [8, 11].

In particular, to determine the location of traffic measurement, a common practice [12, 13, 14, 11, 15, 16] is to perform traffic measurement near in-kernel software switches such as OpenVSwitch [17], Microsoft VFP [18], and FD.io VPP [19]. This is because software switches are responsible for transferring packets among VMs or containers. Hence, they offer full coverage of in-kernel packet processing, thus enabling accurate measurement. Also, such measurement requires both high performance (to keep up with high-speed traffic) and low resource consumption (to avoid affecting tenant services). Thus, a typical choice is to build the approximate algorithms, i.e., sketches, that provide theoretically sound accuracy guarantees with a small resource footprint [8, 12, 13, 11]. These sketches are competent to measure a variety of traffic statistics ranging from heavy hitters to flow entropies, making them a general and attractive way for in-kernel traffic measurement.

Chunming Wu and Jianshan Zhang are corresponding authors.
979-8-3315-0376-5/25/$31.00 ©2025 IEEE

Why eBPF-based sketches? Sketches are typically realized as the in-kernel modules that are integrated into software switches at the compile time [12, 13, 11]. At runtime, they perform measurement operations to extract user-specified flow data (e.g., per-flow byte counts) from the packets received by software switches and record them in their data structures.

However, this approach fails to cope with two-pronged *dynamics*. (1) Traffic patterns and distributions change over time. The memory allocated to each sketch needs to be flexibly adjusted to maintain the accuracy bounds of sketches, e.g., a sketch that aims to detect DDoS attacks needs to monitor the number of active sources for each destination [20, 21]. When traffic grows, the number of flows increases so that the sketch requires more memory to guarantee high accuracy. However, sketches cannot be modified until administrators recompile them and reboot software switches to activate changes. Modifying sketches at runtime leads to service interruption and is considered unacceptable. (2) Administrators will alter their measurement intents at runtime [22, 23, 24, 25, 26]. Their changes can increase or decrease the number of concurrent sketch instances, or replace one type of sketches with another one to compare their accuracy. In the above approach, unless these changes have already been specified in advance, they cannot be enforced on-the-fly as they need recompilation.

In this paper, our focus is on an emerging approach that implements sketches with the extended Berkeley Packet Filter (eBPF) [27] for dynamic network measurement. Unlike kernel modules, eBPF enables administrators to customize the packet processing logic of sketches in their programs with high-level languages like Python. It inserts sketches into the eXpress Data Path (XDP) hooks [28]. These hooks locate in front of in-kernel software switches and have a complete view of every packet received by the kernel. Most importantly, these hooks allow dynamic deployment of eBPF programs. Hence, eBPF-based sketches can be dynamically reconfigured without recompilation and without interrupting other services. In this way, they are able to cope with traffic dynamics and intent changes at runtime. Given this benefit, the industry and acadamia have proposed several projects that exploit eBPF for dynamic network measurement in recent years [29, 30, 16, 31].

Problem. Directly implementing sketches via eBPF suffers from poor performance in terms of low throughput and high latency. Prior research efforts [12, 14, 11, 16] have indicated that the performance of in-kernel operations of measuring packets remains far from ideal. For example, when computing indexes with hash functions, the throughput of sketches can be reduced by 80% [16]. The root cause is that existing sketches are designed with sound theories about the tradeoff between high measurement accuracy and high resource efficiency, while

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:50 UTC from IEEE Xplore. Restrictions apply.

they lack optimizations on CPU consumption.

There are two possible approaches to addressing limitations. The first approach extends the eBPF instruction set and modifying its verification logic to better support sketch operations. But such changes require intrusive kernel modifications and extensive updates across multiple architectures, compromising verifier stability and making deployment difficult. The second method is to implement sketches as built-in kernel modules to avoid performance degradation. But it eliminates the benefits of eBPF as it needs module recompilation and service reboots, hindering dynamic network measurement impossible.

This paper. We propose Carrera, a library that provides high-performance and dynamic sketch-based network measurement. Carrera embraces a set of optimizations to empower the operations of eBPF-based sketches with high-performance packet processing. More precisely, we first conduct a wide spectrum of microbenchmarks to locate the performance bottlenecks of eBPF-based sketches. According to our analysis results, we propose several domain-specific optimizations corresponding to identified bottlenecks and incorporate them into Carrera as basic components of building sketches.

In short, the novelty of Carrera includes identifying performance bottlenecks specific to eBPF-based sketches through microbenchmarks, and integrating several complementary optimizations into an unified framework that accelerates diverse sketches without user efforts. Unlike prior solutions that address isolated bottlenecks, Carrera aims to optimize the entire sketch workflow from packet parsing (O1) to data collection (O5). This helps transform eBPF-based sketches from flexible but inefficient tools into practical solutions for high-speed networks. We list the contributions of Carrera as follows.

- **Microbenchmark-driven bottleneck analysis**. We identify performance bottlenecks in eBPF-based sketches and pinpoint costly operations that degrade throughput and latency. Our findings show that hashing, randomizing, and collecting operations significantly increase the CPU consumption.
- **Domain-specific optimizations**. According to our analysis, Carrera derives: (1) hardware offloading for parsing/hashing operations, (2) SIMD-accelerated parallel hash computation, (3) traffic-aware index caching for skewed workloads, (4) prefetched randomization with shared memory, and (5) active collection of query-relevant sketch data, to collectively reduce CPU cycles and accelerate eBPF-based sketches. The above optimizations well match our findings: the first three optimizations target hashing operations while another two target randomizing and collecting operations, respectively.
- **Testbed evaluation**. We have implemented state-of-the-art types of existing sketches with Carrera and quantified their performance with testbed experiments. Our experimental results demonstrate significant performance benefits of Carrera, including a 20%$\sim$65% improvement in throughput and a 39%$\sim$93% improvement in measurement accuracy.

II. Background and Motivation

A. Sketch-based Network Measurement

In this paper, we follow the line of flow-level network measurement that monitors traffic handled by software switches [12, 13, 14, 11, 15, 16]. Such measurement defines each flow as a sequence of packets using a unique flow ID (e.g., a specific 5-tuple) and computes flow data (e.g., per-flow byte counts) based on the sequence. We focus on a common type of measurement algorithms, i.e., sketches, which outperform the classic method of sampling with both high accuracy and low resource usages [32, 33]. Backed by sound theories, sketches record the data of each packet in their *compact* data structures. They bound their measurement accuracy under the fixed resource capacity, making them readily deployable in commodity measurement points such as software switches.

Without loss of generality, a sketch comprises a data structure that records flow data, an updating algorithm that updates the data structure, and a query algorithm that answers user queries with respect to flow data [32, 34, 35, 25]. (1) The data structure comprises N counter matrixes. The i-th matrix has m_i counter arrays, each of which has n_i fixed-size counters. Each counter can be used to count per-flow packets and bytes or store the flow ID. (2) When a packet arrives at the sketch, the updating algorithm determines which operations to measure the packet with the data structure. More precisely, these operations include parsing packet headers to obtain the flow ID, hashing the flow ID to obtain an index that addresses a specific counter in an array, generating a random number to determine how to manipulate the target counter (e.g., whether to increase or decrease the value [36]), and manipulating the data in the target counter. This algorithm works near software switches to measure every packet in the kernel. (3) The query algorithm runs in the user space and interacts with users. It periodically collects flow data from the data structure. We refer to the time interval between two collections as a measurement *epoch* (e.g., 100 ms [11]). Next, the algorithm transforms flow data into traffic statistics such as heavy hitters. It replies user queries with respect to these statistics.

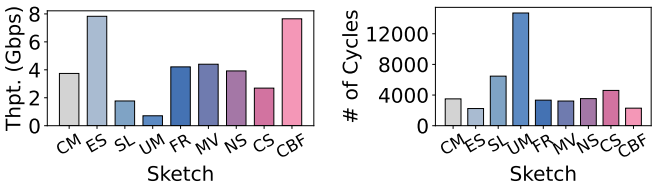
In addition to high accuracy and low resource consumption, which are inherently guaranteed by sound sketch theories, we consider another two requirements in real-world scenarios.

- **Dynamic measurement**. Sketch-based network measurement needs to cope with two-pronged dynamics at runtime [23, 22, 37, 24, 25]. First, when traffic patterns change, the measurement should enable users to dynamically schedule sketch resources, e.g., increasing the amount of memory allocated to sketches. Second, it should allow users to alter their intents on-the-fly, e.g., replacing a sketch with a new one or inserting more types of sketches.
- **High performance**. The measurement should avoid high CPU consumption and provide high throughput and low latency in packet processing [12, 11, 14, 16]. Otherwise, it incurs non-trivial computational overheads in terms of massive CPU cycles. Such overheads translate to low throughput and high per-packet processing latency, failing to cope with traffic workloads. Also, they inevitably affect tenant services on the same servers and reduce the overall revenue.

B. Limitations of Software-based Sketches

Prior research efforts [12, 11, 14, 13] propose to implement sketches in independent kernel modules and insert them into software switches. Doing so empowers them with a complete view of packets handled by software switches,

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:50 UTC from IEEE Xplore. Restrictions apply.



(a) Throughput. (b) # of CPU cycles.

Figure 1: Performance of software-based sketches.

thus guaranteeing full measurement coverage and achieving high accuracy. We refer to these sketches as *software-based sketches*. However, software-based sketches fail to address the above two-pronged requirements by design.

Limitation 1: Poor performance. The performance of most software-based sketches remains low and cannot cope with high traffic workloads [12, 11, 14, 38]. The root cause is that the operations of sketches such as hashing are computationally expensive and lack optimizations on CPU consumption. Therefore, activating these sketches not only slows down the speed of packet processing within software switches but also competes for scarce CPU resources with tenant services, leading to revenue loss. Prior studies [12, 11, 14] indicate that such sketches consume thousands of CPU cycles per packet. Such high consumption translates to poor packet processing performance in terms of low throughput of a few Gbps and high per-packet latency of hundreds of microseconds.

As an illustration, we implement a software-based sketch adopted by data center networks, i.e., FlowRadar [33], with a kernel module in a server. FlowRadar maps packets to its counters and encodes flow IDs with XOR operations. In this way, it enables accurate recovery of flow IDs by countinously XOR-ing counter values with known flow IDs. We quantify the CPU consumption of measuring packets with FlowRadar using Perf [39]. More settings are detailed in §V. Figure 1 indicates that FlowRadar consumes at least 3 K CPU cycles for each packet, which translates to low throughput. We also evaluate other types of sketches and the results are similar. These results comply with prior studies [12, 11] and indicate the poor performance of software-based sketches.

Limitation 2: Lack of dynamics. In view of poor performance, recent work [12, 11] proposes to profile the CPU consumption of each sketch operation and enforce corresponding optimizations with respect to profiling results to accelerate software-based sketches, e.g., reducing the number of hashing operations. However, they still encounter a dilemma that they inherently lack support for the aforementioned dynamics. The reason is that they run inside the kernel modules in software switches while modifying them to cope with dynamics requires recompiling kernel modules with software switches. As a result, software switches have to be rebooted once sketches are modified, thus leading to unacceptable service interruptions. In this context, software-based sketches are considered incapable of handling dynamics on-the-fly.

To validate our claim, we consider a common scenario of handling dynamics when employing software-based sketches. At first, we allocate 10 MB memory to FlowRadar and insert it into the data path of OpenVSwitch. Next, we consider an intent that increases the memory of FlowRadar to 100 MB [34]. Since the data structure of FlowRadar defined in a kernel

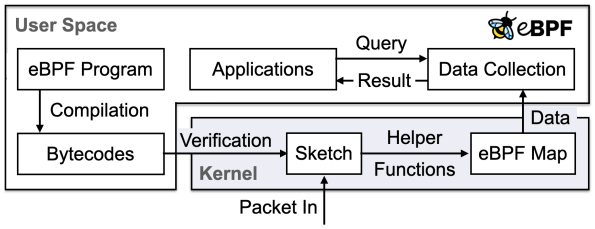


Figure 2: Overview of eBPF-based sketches.

module is fixed at runtime, we need to stop the module, modify it with new memory capacity, and recompile it to enforce the intent. As a result, OpenVSwitch needs to be shut down for a while to deploy our changes, leading to service downtime. We measure such downtime and repeat our measurement for 100 times. Our results show that the downtime varies from tens of seconds to a few minutes, which inevitably affects tenant services (e.g., interrupting the connections that target VMs due to switch shutdown) for a long time.

There exist two possible workarounds. First, one may argue that sperating the kernel modules that realize sketches from software switches, e.g., realizing sketches with NetFilter hooks [40] invoked before software switches, can prevent switches from experiencing downtime. However, this method brings new risks to the entire system. This is because any bugs or abnormalities in sketches can make the kernel crash. So this method is a dangerous endeavor and is thus disqualified from consideration in production networks.

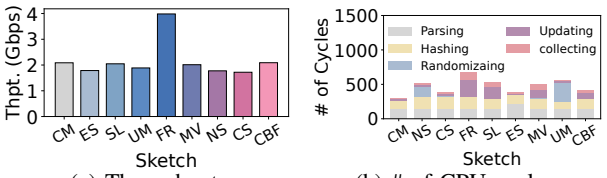
Another workaround that can eliminate the need of sketch recompilation and switch downtime is to employ multiple concurrent sketch instances, which use different amounts of memory. At runtime, an instance acts as the primary while others are backups. If users require their sketch to use more resources, they can switch the roles of the primary sketch and a backup that has more memory. Hence, this primary-backup approach reacts to dynamics without downtime. Nevertheless, it comes with the cost of significant CPU consumption due to its excessive use of multiple sketches. So it is impractical.

C. Pros and Cons of eBPF-based Sketches

eBPF-based sketches. The emergence of eBPF [27] provides new opportunities of simultaneously achieving high performance and handling runtime dynamics. More precisely, eBPF enables its users to customize sketches in eBPF programs using high-level programming languages such as C and Python. Then BCC [41] compiles and transforms eBPF programs into eBPF bytecodes. These bytecodes are then verified to determine if they can be safely loaded on the kernel space. If they pass verification, they are just-in-time compiled into native machine codes attached to target hooks in the kernel. In this way, eBPF-based sketches can run independently of software switches while ensuring safety. Also, their fast just-in-time compilation well copes with runtime dynamics.

Figure 2 summarizes an overview of eBPF-based sketches. These sketches maintain their data structures in the eBPF map in the kernel space. The eBPF map can be dynamically altered to satisfy user-specified intents of manipulating sketch data structures (e.g., adding more counters, arrays, and matrixes). At runtime, eBPF-based sketches perform their updating algo-

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:50 UTC from IEEE Xplore. Restrictions apply.



(a) Throughput. (b) # of CPU cycles.

Figure 3: Performance of eBPF-based sketches.

rithms that are realized with eBPF helper functions, each of which corresponds to an atomic operation (e.g., writing data to the eBPF map), in the kernel space. In addition, the user space activates the query algorithm that periodically collects data from the eBPF map and processes data in service of user queries. The two types of algorithms run asynchronously.

Performance evaluation. Athough eBPF-based sketches are able to handle runtime dynamics, we observe that they suffer from poor packet processing performance. In detail, we select five representative sketches, including the Count-Min sketch (CM) [42], elastic sketch (ES) [13], FR [33], SketchLearn (SL) [43], and UnivMon (UM) [8], and implement their data structures and algorithms using eBPF. We run sketches on the testbed described in §V-B. We run PktGen [44] on the tester to replay a CAIDA 2019 trace [45] to generate traffic workloads for testing eBPF-based sketches.

Figure 3 presents the average throughput and CPU consumption of eBPF-based sketches after 100 runs. These results validate our claim: even the most performant sketch, CM, spends a lot of CPU cycles per packet, leading to the poor throughput of $<$5 Gbps and high latency of several milliseconds. Such poor performance renders the benefits of eBPF-based sketches in practice.

D. Goals and Key Ideas

Goals. This paper focuses on in-kernel sketch-based traffic measurement, which is one of the main concerns in modern networks [12, 13, 11, 16]. We aim to investigate the design space that enables in-kernel sketch-based measurement with the characteristics of dynamics and high performance (§II-A).

Key ideas. To achieve our goals, we embrace two-pronged key ideas. First, to achieve dynamic measurement, we follow the line of eBPF-based sketches that are naturally able to handle dynamics given the advantages of eBPF. Second, to achieve high performance, we first conduct microbenchmarks to study the performance of atomic eBPF actions that collectively form sketch operations. According to the results, we identify the performance bottlenecks that prevent eBPF-based sketches from being high-performance. Moreover, for each bottleneck, we identify the root cause and propose a set of corresponding optimizations to mitigate the bottleneck for arbitrary eBPF-based sketches. Finally, we integrate these optimizations into a single framework, namely Carrera, that encapsulates them into a suite of intuitive APIs. These APIs allow users to build performant eBPF-based sketches in a high-level manner.

III. Microbenchmarks

We perform extensive microbenchmarks to locate the performance bottlenecks of eBPF-based sketches.

Tested sketch operations. According to the literature [46, 11], we summarize the following types of sketch operations that widely exist in existing sketches.

1 **Parsing**. This operation parses packet headers and combines the fields of interest into a flow ID. A typical example is to extract source and destination IP addresses, source and destination layer-4 ports, and IP protocol from the packet to assemble the 5-tuple flow ID.
2 **Hashing**. This operation receives the flow ID as input and invokes hash functions to compute indexes by hashing the flow ID. These indexes address counters in the sketch.
3 **Randomizing**. This operation acquires one or more random numbers. These numbers determine how to update sketch counters, e.g., increasing or decreasing counter values [8].
4 **Updating**. This operation updates the values in the sketch counters addressed by indexes. It varies across sketches, e.g., it simply increases the value by one in the Count-Min sketch [42], while in UnivMon [8], it randomly chooses to increase or decrease the counter by one.
5 **Collecting**. This operation periodically retrieves sketch data from sketch counters to the user space for answering user queries. Network management applications rely on this data to monitor flows of interest, such as detecting DDoS attacks or identifying heavy hitters. The collection occurs at the end of each measurement epoch, where a user-space process reads the entire sketch data structure.

Results. Our microbenchmarks employ the same sketches and experimental settings as §II-C. For each sketch, we decompose it into five sets, each comprising one specific type of the above operations. Then our microbenchmarks conduct three steps. (1) We execute an empty eBPF program, which only receives and forwards packets, and measure the average number of CPU cycles processing each packet as the baseline λ_0. (2) We generate four eBPF programs, each of which activates only one type of sketch operations. We activate only one program at a time while measuring the consumed number λ' of CPU cycles per packet. We obtain the CPU cycles consumed by each set of operations via $\lambda' - \lambda_0$. (3) We run the entire sketch and monitor the overall number λ of CPU cycles per packet. Then we can quantify the impact of each set of operations on sketch performance via $(\lambda' - \lambda_0)/\lambda * 100\%$.

Figure 3(c) summarizes the results. It indicates that *hashing, randomizing, and collecting operations contribute significant portions to the overall number of CPU cycles*. Specifically, for sketches with randomizing operations, the hashing, randomizing and collecting operations account for 23%, 37% and 6% of the total processing cycles. In contrast, for those without randomizing operations, hashing and collecting operations account for 33% and 7%, respectively. The reasons are threefold. (1) Each sketch requires several indexes to locate many counters, which reduces the probability of hashing collisions when recording a flow. As a result, it needs to hash the flow ID multiple times to compute indexes, which occupies a lot of CPU cycles. (2) Realizing randomizing operations in eBPF depends on specific eBPF helper functions, which are inefficient by nature. (3) The passive collection approach incurs high overheads because it transfers all counters regardless of their relevance to user queries or whether their values have changed since the last epoch. Hence, these results guide us to

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:50 UTC from IEEE Xplore. Restrictions apply.

Table I: Applicability of Carrera on existing sketches.

Network Management Application	Sketch Name	Applicability of Carrera
Frequency Estimation /Heavy Hitters	Count-Min [47]	O1, O2, O3, O5
	Count Sketch [48]	
Hierarchical Heavy Hitters	RHHH [49]	O1, O2, O3, O5
	HHH [50]	
Cardinality	PCSA [51]	O1, O2
	MRB [52]	
	LogLog [53]	
	HyperLogLog [9]	
Entropy	EntropySketch [54]	O1, O2, O3, O5
Change Detection	K-ary [55]	O1, O2, O3, O5
Super Spreaders	SpreadSketch	O1, O2, O3, O5
General	FCM [56]	O1, O2, O3, O5
	SketchLearn [43]	
	ElasticSketch [13]	
	UnivMon [57]	O1, O2, O3, O4, O5
	NitroSketch [11]	

derive optimizations towards sketch operations.

IV. Carrera Design

A. Overview

Optimizations. Carrera employs five major optimizations. It employs O1, O2, and O3 for optimizing hashing operations, utilizes O4 to reduce the CPU cycles consumed by randomizing operations, and uses O5 to improve sketch data collection.

- **O1: Sketch operation offloading**. Carrera offloads some sketch operations to programmable hardware such as programmable switches and data processing units (DPUs). Hardware offers orders-of-magnitude higher performance than general-purpose CPU cores in packet processing, e.g., Tbps-level throughput and microsecond-level latency in programmable switches [58]. As such, such offloading brings remarkable performance improvements to sketches.
- **O2: Hashing acceleration**. For each sketch, Carrera identifies which type of hash functions consumes the minimum number of CPU cycles and best matches the sketch without obvious measurement accuracy loss. It equips the sketch with the fastest hash function to boost hashing. Also, it leverages single instruction multiple data (SIMD) instructions to parallelize the computing of multiple hash functions when addressing sketch data structures.
- **O3: Index caching**. Given the skewness of real traffic [34, 45], i.e., a few large flows contribute most packets in traffic, the number of distinct flow IDs is small, i.e., most packets have the same flow IDs. Hence, after hashing, the number of distinct indexes is also small. By caching the indexes of large flows, Carrera bypasses hashing operations for most packets to accelerate sketch processing.
- **O4: Random number prefetching**. For the sketch operations utilizing randomized updating, e.g., updating the indexed counters with random values, Carrera discards the current practice that generates random numbers using eBPF helper functions, which is costly. Instead, it adopts the strategy that installs a user-space process to populate the random numbers generated in advance to sketches. Thus, sketches only need to lookup random numbers rather than generating them, which obtains performance benefits.
- **O5: Active sketch data collection**. Carrera replaces passive data collection with an active approach. Instead of transferring the entire sketch data structure at each epoch, Carrera dynamically pushes only the non-zero changes of counters associated with user-specified flows. This reduces both the volume of transferred data and consumed CPU cycles.

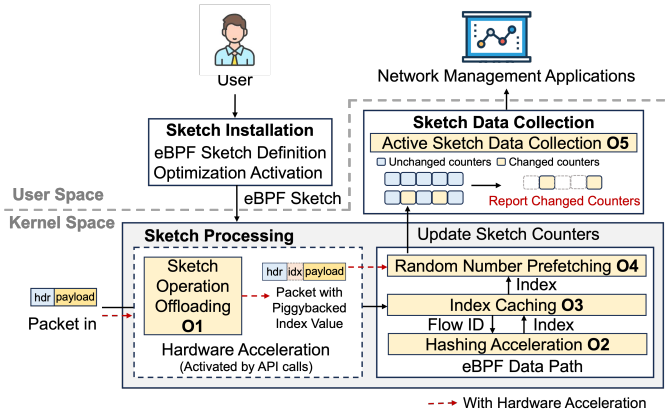


Figure 4: Carrera overview

In particular, our microbenchmarks in §III show that hashing, randomizing, and collecting operations contribute significant portions to the overall number of CPU cycles. Hence, our optimizations directly match our findings, i.e., O1-O3 for optimizing hashing operations, O4 for optimizing randomizing operations, and O5 for optimizing collecting operations.

Table I summarizes the applicability of these optimizations against different network management applications and corresponding sketches. Since we design Carrera with respect to our microbenchmarks towards general sketch operations, Carrera's optimizations can be applied to a wide range of sketches.

Workflow. Carrera executes a three-step workflow (Figure 4).

- **Step 1: Sketch installation**. Users define sketches in eBPF programs as usual. When submitting their sketches, Carrera allows them to choose which optimizations to activate via four APIs corresponding to O1-O4. Here, O1 is activated only when users invoke the API, while other optimizations are activated by default.
- **Step 2: Sketch processing**. If O1 is activated, packets will first go through the hardware to be handled by offloaded sketch operations. Their intermediate processing metadata (e.g., indexes) will be piggybacked on packet headers. Then packets are directed to the non-offloaded part of sketches, which runs in the kernel through eBPF. This part completes measurement towards these packets based on piggybacked metadata while utilizing O2-O4 to accelerate its processing. If O1 is not activated, packets will be sent directly to eBPF-based sketches, which execute O2-O4.
- **Step 3: Sketch data collection**. At runtime, sketches keep track of changes to counters corresponding to user queries. At the end of each measurement epoch, Carrera pushes only these changes to the user space. Network management applications directly consume the changes to answer queries, eliminating the need for full data transfers.

B. O1: Sketch Operation Offloading

Insight. Our first insight comes from the recent emergence of programmable hardware, including programmable switches and DPUs. These types of hardware can provide orders-of-magnitude higher performance than the end-host CPU cores

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:50 UTC from IEEE Xplore. Restrictions apply.

that execute eBPF programs in packet processing. For example, programmable switches guarantee O(10 Tbps) throughput and O(1 μs) per-packet processing latency regardless of the complexity of packet processing operations. Thus, a natural solution is to offload as many operations as possible from eBPF to the hardware to save CPU cycles.

Challenge. The offloading yields the challenge of coordinating the offloaded operations in hardware and non-offloaded ones in eBPF. In detail, sketch operations are interdependent, e.g., parsing operations provide the flow ID to hashing operations and thus should be executed first. As such, the results of previously executed operations should be passed to subsequent operations to guarantee correct sketch execution.

Also, hardware has strict resource limitations, e.g., a programmable switch has less than 15 MB SRAM. This requires the offloading to carefully deploy operations in hardware without saturating the hardware resource capacity.

Solution. Carrera employs two strategies to tackle the above challenges. First, to enable the coordination, it piggybacks the intermediate results of previously executed operations on each packet. In detail, the hardware inserts a specific header into each packet. This header records the processing results of offloaded operations. When the hardware completes its processing, the packet with piggybacked results is transferred to non-offloaded operations, which extract the results to continue their processing. Second, Carrera encodes the limitations of hardware resources as constraints while following existing studies [59, 21] to formulate its offloading as an optimization problem. It optimally solves the problem to make resource-efficient offloading decisions.

Carrera performs three steps to integrate these strategies.

Step 1: Offloading. Carrera first chooses which types of operations to offload. Since offloading sketch data structures may saturate hardware resources, it only offloads parsing and hashing operations by default since these operations do not need to be deployed near sketch data structures. Next, it formulates an optimization problem, where the constraints consider the budget of switch resources available while the objective is set to minimizing hardware resource consumption. Solving the problem returns the decision variables that indicate how to deploy operations with hardware resources. According to these variables, Carrera offloads the operations.

Step 2: Piggybacking. Carrera installs an additional module that is located next to offloaded operations. This module inserts a header to each packet. It writes the processing results of offloaded operations, i.e., indexes, to the header.

Step 3: Coordinating. Carrera transfers each packet from the offloaded operations in hardware to the non-offloaded ones in eBPF. When a packet is received by non-offloaded operations, the indexes piggybacked in each packet are input to non-offloaded operations to normally execute measurement.

C. O2: Hashing Acceleration

Insight. A sketch usually invokes multiple hashing operations to compute several indexes for updating their data structures. For example, the count-min sketch uses k hashing operations to compute k different indexes. It utilizes these indexes to locate and update k diverse counters, which reduces collision. Therefore, we have an opportunity to accelerate hashing operations via SIMD-based execution parallelism in the cases when hardware offloading is not available.

Challenge. The eBPF instruction set does not provide any support for SIMD-based computing. If Carrera wants to utilize the benefits of SIMD when computing indexes via multiple hashing operations, it has to invoke low-level SIMD interfaces in eBPF programs. This leads to two issues. First, an index has at least 32 bits in its size. Hence, multiple indexes may be too large to be populated to 256-bit SIMD registers. As a result, SIMD also needs several rounds, each returning a few indexes, to complete the execution, forfeiting its performance benefits. Second, the results of SIMD will be copied twice. They are first copied from SIMD registers to eBPF memory, and are then copied from eBPF memory to the eBPF registers that are accessed by sketch operations. This further compromises the performance benefits of SIMD.

Solution. To maximize the benefits of SIMD, Carrera chooses to collectively execute hashing operations and subsequent updating operations, bypassing the need of copying indexes between memory and registers. In detail, we observe that the indexes, which are supposed to be produced by SIMD, are *intermediate results* in sketches. They are only used to address counters during sketch processing, while the final measurement results are counter values rather than indexes. Motivated by our observation, Carrera integrates the hashing operations into updating operations in sketches when activating SIMD. In detail, it performs the following steps.

Step 1: Input. Carrera inputs the required parameters, including the pointers of sketch counter arrays and the flow ID, to SIMD-accelerated hashing and updating operations.

Step 2: Hashing. SIMD populates input parameters into its registers and executes multiple hashing operations in parallel. The results, i.e., indexes, are stored by SIMD registers.

Step 3: Updating. Carrera retrieves indexes from SIMD registers to locate and update sketch counters. This eliminates the original workflow that copies indexes to eBPF memory and registers, retaining SIMD performance benefits.

D. O3: Index Caching

Insight. Network traffic exhibits high skewness. By skewness, we mean that while there are numerous possible flows (e.g., 2^{64} 2-tuple flows), only a few flows have a large number of packets. This has been justified in many measurement studies [34, 60] and used in various recent solutions [12, 13, 34]. As such, most received packets share the same flow IDs, such that the number of distinct flow IDs is small. Hence, we can directly cache the indexes of hot flow IDs. In this way, most packets that contain the hot flow IDs will hit the cache and bypass the time-consuming hashing operations, reducing their processing time and improving overall sketch performance.

Challenge. Constructing the cache involves the design of two algorithms. The first algorithm identifies hot flow IDs, while the second one populates the indexes of hot flow IDs to the cache and removes stale indexes from the cache. If we implement the two algorithms on the eBPF data path that executes sketches, the overheads incurred by the two algorithms during packet processing may overwhelm the benefits of prior

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:50 UTC from IEEE Xplore. Restrictions apply.

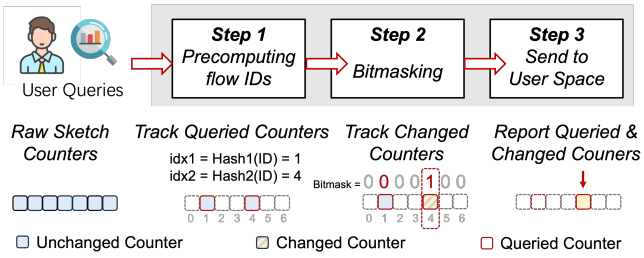


Figure 5: O5: Active sketch data collection

optimizations. Hence, how to realize these algorithms while avoiding high overheads challenges our cache management.

Solution. Carrera proposes to manage the cache of indexes of hot flow IDs through a user-space process. More precisely, Carrera leverages a crucial characteristic of eBPF that eBPF maps are shared memory between user-space processes and kernel-space packet processing. Thus, it builds the cache with eBPF maps while collaborating the eBPF data path that runs sketches and the cache and a user-space process to collectively manage the cache. We summarize the workflow as follows.

Step 1: eBPF data path. When the eBPF data path receives a packet, it first parses the packet to retrieve the flow ID. Then it lookups the cache with the ID. If the ID does not exist in the cache, it reports the ID to the user-space process. Otherwise, it reports the visit frequency of the cache entry, i.e., how many packets visit the entry in the current measurement epoch.

Step 2: User-space process. The user space continuously receives flow IDs from the eBPF data path. It sorts the flow IDs in descending order of their visit frequency. For the top flow IDs, it computes the indexes by hashing them, and inserts indexes to the cache built by a shared eBPF map. For the IDs that are previously identified as hot but now become stale, it removes these IDs from the cache.

Step 3: Cache. In the eBPF data path, the cache is executed before hashing operations. Each incoming packet will first lookup its flow ID in the cache. If the packet matches one cache entry, Carrera retrieves the indexes from the entry and directly utilizes them in subsequent updating operations, which bypasses hashing operations. Otherwise, the packet is normally processed by all sketch operations.

E. O4: Random Number Prefetching

Insight. eBPF has an inherent helper function that is dedicated to generating random numbers. Unfortunately, this function is inefficient and consumes a lot of CPU cycles. For the sketches that need to generate random numbers for each packet, e.g., NitroSketch [11], generating random numbers prevents them from being performant. Thus, one workaround is to generate a lot of random numbers in advance. When a packet arrives, sketches can immediately acquire random numbers instead of generating these numbers from scratch, which saves CPU cycles and improves performance.

Challenge. Realizing this insight brings two-pronged challenges. First, directly generating random numbers in the eBPF data path inevitably introduces high CPU consumption, which may affect sketch processing. Second, for the sketches that use randomizing operations, the measurement of each packet will consume random numbers. As such, the eBPF data path needs to continuously generate random numbers to support sketch operations. Satisfying this requirement occupies a non-trivial portion of computational resources.

Solution. In response, Carrera employs the same strategy as O3, i.e., leveraging the user-space process to generate random numbers in advance. Its workflow is summarized as follows.

Step 1: Random number pool. Carrera creates a random number pool with an eBPF map, which is a portion of memory shared between the user space and eBPF-based sketches.

Step 2: Random number generation. In the user space, a process continuously generates random numbers while populating these numbers into the pool.

Step 3: Random number retrieval. In the eBPF data path, the randomizing operations of sketches continuously retrieve random numbers from the pool. Once a number is retrieved, it is removed from the pool to ensure randomness.

F. O5: Active Sketch Data Collection

Insight. User queries typically target a small subset of flows while many sketch counters remain unchanged between consecutive epochs. Collecting only the changed counters relevant to user queries reduces data transfer volume and CPU consumption. For instance, DDoS detection focuses on suspicious flows, whose counters are sparse in the sketch. Actively pushing these changes helps sketch data collection match query needs while bypassing redundant data.

Challenge. When realizing this approach, we face two issues. First, since sketches comprise millions of counters, tracking counter changes may introduce significant per-counter metadata overheads. Second, mapping the flow IDs corresponding to user queries to counters in real-time requires additional time and may degrade packet processing performance.

Solution. In response, Carrera adopts a three-step workflow.

Step 1: Precomputing flow IDs. Carrera identifies the flow IDs specified by user queries (e.g., 5-tuples of suspicious flows) and precomputes their corresponding sketch indexes using the same hash functions as the sketch. This ensures that the mapping between flows and counters is consistent and avoids recomputing hashes during packet processing.

Step 2: Bitmasking. Carrera adopts a lightweight bitmask structure in eBPF maps to track whether counters associated with these flows have changed. Each bit in the bitmask corresponds to a counter. The sketch updates the bitmask atomically when modifying a counter. This minimizes metadata overheads while enabling efficient change detection.

Step 3: Collection. At the end of each epoch, Carrera only pushes the changed counters marked in the bitmask to the user space. A user-space process fetches modified counter values and answers user queries.

G. Discussion

Support of O1. Existing solutions such as FITS [16] cannot directly leverage O1 because they operate strictly with the eBPF runtime on the host's CPU. Also, they lack mechanisms to coordinate with programmable hardware or deliver intermediate results (e.g., indexes) from hardware to eBPF. Carrera overcomes this by setting hardware to precompute indexes and piggyback them on packets. These indexes are reused by eBPF for subsequent operations.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:50 UTC from IEEE Xplore. Restrictions apply.

Table II: Latency overheads of index piggybacking.

Size of Indexes	4 bytes	8 bytes	12 bytes	16 bytes	20 bytes
Latency (ns)	8 ns	10 ns	8 ns	11 ns	12 ns

In Carrera, O1 depends on hardware capabilities. For DPUs, their SoC cores support arbitrary types of hash functions since they are general-purpose. For programmable switches, current Tofino2 supports integrating arbitrary hash functions to ASIC pipelines via "extern" primitives. So we set Carrera to use the original hash functions used by sketches by default. However, Tofino1 only supports CRC hashing functions, making Carrera 's offloading limited. We list this limitation as our future work.

Next, we measure the resource consumption when O1 is activated. We extract detailed reports from switch compilers: O1 consumes at most 4.9% packet header vectors and 4.7% very long instruction words without consuming other resources during our evaluation in §V, which is acceptable.

Piggybacking overheads. Carrera piggybacks all indexes on the packet. This will increase packet length and thus affect the latency. However, given the limited size of each index, e.g., 4 bytes, the impact is negligible. In detail, we fix the packet size to 700 bytes, which is close to the average in real traffic traces [45]. We use hardware timestamps to measure the per-packet latency with and without piggybacking while computing their average differences after 100 runs as the overhead. We vary the number of piggybacked indexes from 1 to 5, i.e., adding 4-20 bytes in each packet. Our results in Table II present that piggybacking only adds up to 10 ns to the overall latency.

User-kernel coordination. Carrera avoids its coordination between user space and kernel space from becoming bottlenecks by limiting the frequency of coordination. For example, O3 needs to write the cached indexes from the user space to eBPF. Since such coordination is completely user-space-driven, Carrera can easily throttle the frequency of coordination. Even under high-frequency update scenarios, it allows users to make the tradeoff between updating frequency and possibly incurred overheads, e.g., users set a low frequency to avoid performance bottlenecks at the cost of relatively slow updating.

V. Evaluation

A. Implementation

We have implemented two versions of Carrera, including Carrera-O1 (i.e., Carrera with O1 activated) and Carrera (i.e., by default, Carrera disables O1 for a fair comparison).

Offloading. We have implemented O1 of Carrera on a Tofino switch and an NVIDIA BlueField3 DPU, respectively. Carrera offloads the parsing and hashing operations of target sketches. It leverages Gurobi to determine the resource-optimal offloading decisions. For Tofino switches, offloaded operations are realized in a data plane program compiled to switch ASIC pipelines. They exploit underutilized computational resources, including ALUs and very long instruction words (VLIWs). For DPU, these operations utilize eight 2.75 GHz Arm A72 SoC cores (shared 6 MB L3 cache, 32 GB DDR4) via DOCA.

eBPF implementation. We have implemented O2-O5 of Carrera on a commodity server. (1) We implement O2 by building SIMD-based hash operations as a kernel module due to the lack of SIMD support in eBPF program. These operations are exposed to eBPF program through the *kfunc* mechanism. (2) For O3, we execute a user-space process that manages the cache of indexes corresponding to hot flow IDs. The cache is an eBPF map shared between the user-space process and kernel-space sketches. When the frequency indicates that the ID belongs to the top-k most visited IDs, the ID is inserted to the cache. Otherwise, if the ID exists in the cache, the process removes the ID from the cache since the ID becomes cold. By default, we configure k to 2^{16}. (3) We implement O4 using *bpf_map_update_elem*, a queue data structure native to eBPF. A user space process continuously populates the queue with random numbers while eBPF sketches retrieve these numbers as needed for operations. (4) O5 is realized similarly as O4. Also, Carrera limits each sketch to only use one eBPF map in its data structures to minimize memory calls.

Table III: Comparison between Carrera and existing solutions

Characteristics	FITS [16]	eNetSTL [64]	P4/SDN OpenFlow	Carrera
Microbenchmark	✗(Ad-Hoc)	✗	✗	✓
Integration of Sketch-Specific Optimizations	Partial (like O2+O4)	✗ (specific to NFs)	✗	✓
Offloading	✗	✗	✓(switch)	✓(switch, DPUs)
Traffic-Aware Caching	✗	✗	✗	✓
Active Collection	✗	✗	✗	✓
Runtime Reconfig.	✓	✓	✗	✓
Automated and Cross-Sketch	✗(Manual)	✗	✗	✓

B. Methodology

Testbed. We build a testbed that contains two servers directly connected via a 100 Gbps link. Each server is equipped with a 36-core Intel(R) Xeon(R) Gold 6240C CPU (2.60 GHz) and 128 GB RAM. These servers run Ubuntu 22.04 with the 64-bit Linux 6.6 kernel. Both servers are equipped with a 400 Gbps NVIDIA BlueField-3 DPU. One server generates 100 Gbps workload using PktGen and the other server acts as a receiver that runs various sketches to measure incoming traffic.

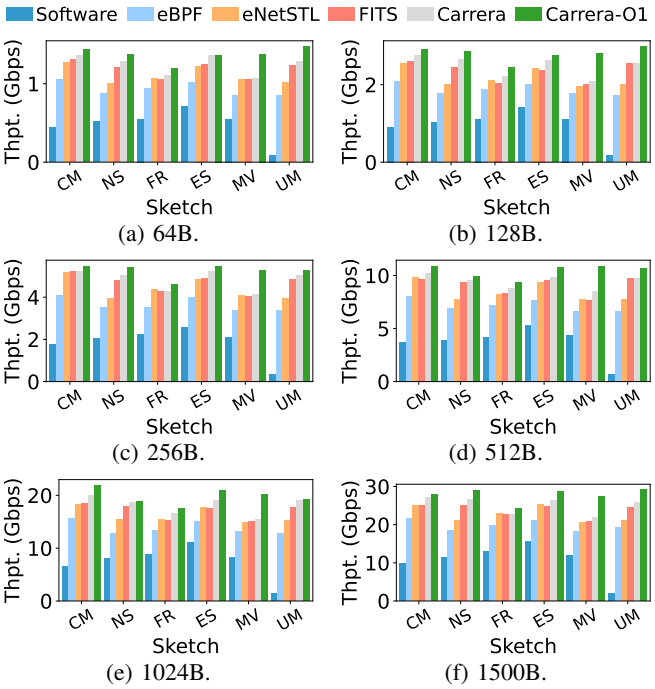
Sketches. We implement six sketches, including count-min sketch (CM), elastic sketch (ES), FlowRadar (FR), NitroSketch (NS), and UnivMon (UM). Each sketch is allocated 10 MB memory. We configure the parameters of each sketch based on the theoretical bounds in their original papers. We set 8 arrays and 8 hash functions for CM and FR. For ES, we allocate 1/4 of total memory for its heavy part and 3/4 for the light part. For UM, we allocate 4 MB memory to its first array and 250 KB for each of the rest of arrays. For NS, we set its geometric sampling rate to 0.01 while selecting parameters based on its 5% accuracy guarantee.

Workloads. We use two different workloads to evaluate Carrera. (1) We use PktGen to generate 100 Gbps traffic with fixed packet lengths. Each packet contains uniformly distributed flow ID. This workload serves to stress-test the performance of Carrera. (2) We use DPDK to replay four real-world traces, including CAIDA 2018, CAIDA 2019 [45], MAWI [61], IMC [62]. Also, we create two traces that follows Zipf [63].

Comparison solutions. We select four comparison solutions.

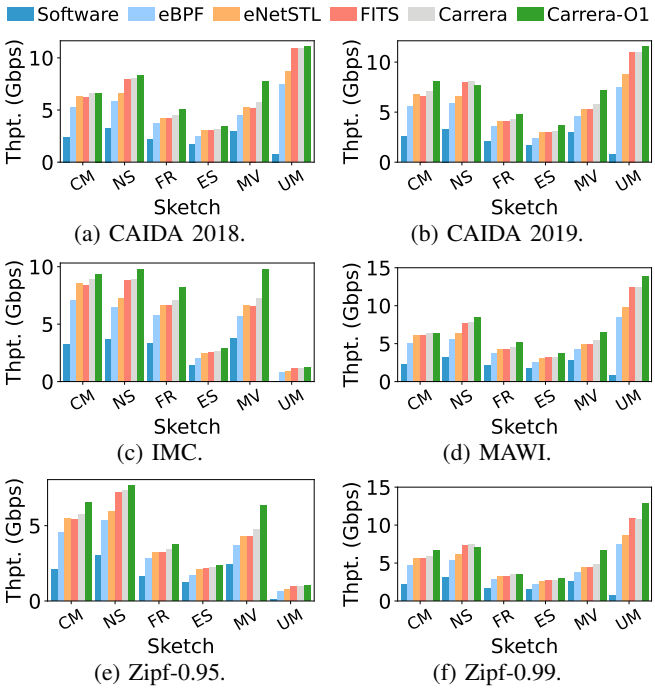
- For software-based sketches, we follow existing studies [12, 11] to realize these sketches using C++ libraries like libpcap.
- For eBPF-based sketches, we realize them in XDP hooks.
- eNetSTL [64] is a framework that aims to accelerate eBPF-based network functions. We use its APIs to realize sketches

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:50 UTC from IEEE Xplore. Restrictions apply.



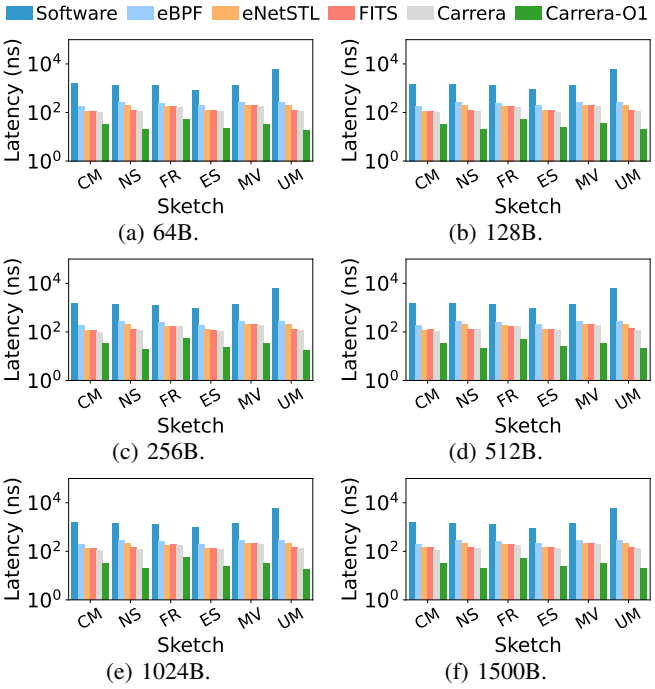
(a) 64B. (b) 128B. (c) 256B. (d) 512B. (e) 1024B. (f) 1500B.

Figure 6: (Exp#1) Throughput.



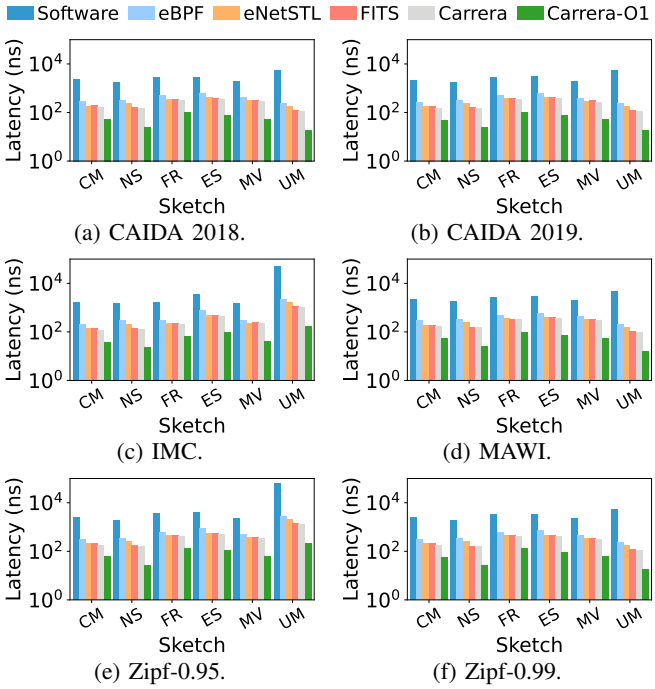
(a) CAIDA 2018. (b) CAIDA 2019. (c) IMC. (d) MAWI. (e) Zipf-0.95. (f) Zipf-0.99.

Figure 8: (Exp#1) Throughput (traces).



(a) 64B. (b) 128B. (c) 256B. (d) 512B. (e) 1024B. (f) 1500B.

Figure 7: (Exp#1) Latency.



(a) CAIDA 2018. (b) CAIDA 2019. (c) IMC. (d) MAWI. (e) Zipf-0.95. (f) Zipf-0.99.

Figure 9: (Exp#1) Latency (traces).

in eBPF, which activates O2, i.e., hashing acceleration.

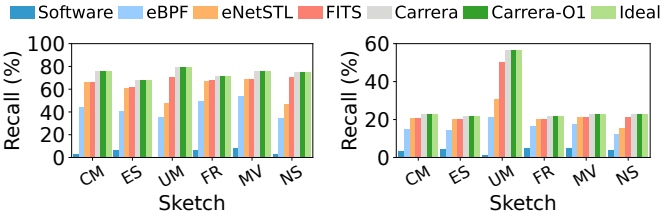
- Fast in-kernel traffic sketching (FITS) [16] embraces some optimization techniques to improve sketch performance. It considers (1) limiting each sketch to only use one eBPF map to minimize memory calls, (2) choosing the fastest functions in hashing operations, and (3) prefetching random numbers (which is similar to O4). Compared to FITS, Carrera is a parallel work and activates more optimizations covering the complete life cycle of eBPF-based sketches (see Table III).

C. Effectiveness of Carrera

(Exp#1) Sketch performance. We evaluate the performance of sketches optimized by Carrera against existing solutions. In our testbed, we first generate 100 Gbps traffic via PktGen and increase per-packet sizes from 64 bytes to 1500 bytes. Then we replay the above traces. We inject traffic to sketches while measuring the throughput. According to Figure 6-9, Carrera outperforms eBPF and software solutions with 20% and 65% higher throughput, and 39% and 93% lower latency.

(Exp#2) Measurement accuracy. We evaluate the measurement accuracy of sketches optimized by Carrera. We use two

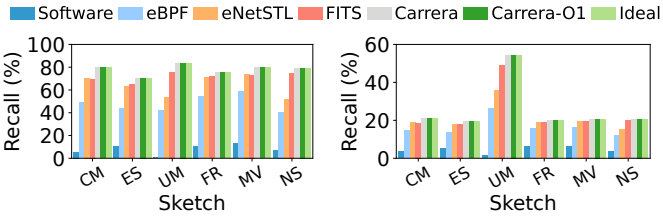
Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:50 UTC from IEEE Xplore. Restrictions apply.



(a) CAIDA 2018. (b) MAWI.

(c) IMC. (d) Zipf-0.99.

Figure 10: (Exp#2) Accuracy (HH).



(a) CAIDA 2018. (b) MAWI.

(c) IMC. (d) Zipf-0.99.

Figure 11: (Exp#2) Accuracy (SS).

applications: heavy hitter detection (HH) and superspreader detection (SS). These applications identify the flows of their interest: the thresholds of HH is 0.1% packets while SS detects the flow, which number of packets exceeds 0.5% of the total number of observed IP addresses. Also, we consider diverse types of traces, including CAIDA, MAWI, IMC, Zipf traces. We employ Carrera and comparison solutions to deploy sketches and use their sketches to measure the target flows within these traces, respectively. We quantify the accuracy of each task via three steps. (1) We directly parse each trace to obtain the correct results. (2) We obtain the measurement results of sketches optimized by Carrera or comparison solutions. (3) We compare the measurement results with the correct results to compute accuracy metrics, e.g., false positives correspond to the flows that are wrongly identified as the flows of interest. According to these metrics, we compute the accuracy in terms of Recall. In Figure 10 and Figure 11, Carrera improves the accuracy by up to 46%. Such benefits come from its higher throughput and lower latency, which enable its measurement to measure every packet in each epoch.

D. Ablation Study

(Exp#3) Offloading benefits (O1). We isolate the benefits of offloading parsing/hashing operations to hardware by comparing Carrera's Tofino-offloaded sketches against original eBPF implementations. In Figure 12, offloading can reduce the per-packet latency in sketches by 42%-90%. This is because offloading to programmable hardware frees CPU resources, proving O1's effectiveness in alleviating host bottlenecks.

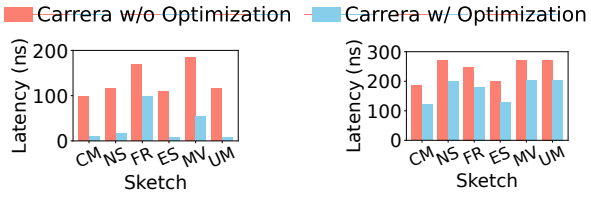


Figure 12: O1.

Figure 13: O2.

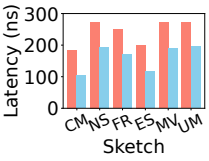


Figure 14: O3.

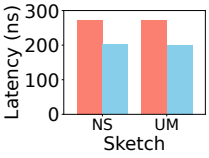


Figure 15: O4.

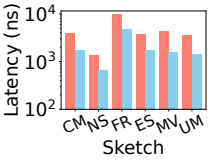


Figure 16: O5.

(Exp#4) Hashing acceleration (O2). We evaluate Carrera's O2 against default eBPF hashing (xxHash). In Figure 13, O2 reduces per-packet latency by 27%, e.g., it completes 8 hashing operations within 24 ns. This helps multi-index sketches avoid long hashing computation delay.

(Exp#5) Index caching (O3). We evaluate O3 under a CAIDA trace (85% of packets from 5% of flows) by comparing it with original eBPF sketches. Figure 14 indicates that O3 reduces per-packet latency by an average of 35%. Since traffic in most real-world scenarios is highly skewed, O3 shows promise in building eBPF-based sketches with robust high performance.

(Exp#6) Random prefetching (O4). We compare O4 against on-demand generation in UnivMon [57] and NitroSketch [11]. In Figure 15, when compared to original sketches, O4 removes unnecessary randomizing operations at runtime and leads to the average latency reduction of 26%.

(Exp#7) Active data collection (O5). We compare Carrera's active sketch data collection (O5) against passive approaches to quantify performance gains. We consider the user queries that monitor 0.1% of all flows. For passive collection, a user-space process reads all sketch counters every epoch (100 ms), while O5 direcly tracks the changed counters linked to queried flows. In Figure 16, O5 reduces collection latency by 56% because it significantly reduces the amount of transferred data.

VI. Conclusion

We propose Carrera, a library that offers high-performance eBPF-based sketches in network measurement. We analyze the performance bottlenecks in eBPF-based sketches via microbenchmarks. Then we design domain-specific optimizations in Carrera, i.e., sketch operation offloading, hashing acceleration, index caching, random number prefetching, and active data collection. Testbed results indicate that Carrera improves throughput by up to 65% and reduces latency by up to 93%.

Acknowledgement. We thank our reviewers and our shepherd for their constructive comments. This work is supported by the National Key R&D Program of China (2023YFB2904005), the National Natural Science Foundation of China (623B2090, 62172007, 61902362), Chinese State Key Laboratory of Industrial Control Technology (ICT2025A13), and the Fundamental Research Funds for the Central Universities (2025ZFJH02).

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:50 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] T. Koponen, K. Amidon, P. Balland, M. Casado, A. Chanda, B. Fulton, I. Ganichev, J. Gross, P. Ingram, E. Jackson *et al.*, "Network virtualization in multi-tenant datacenters," in *USENIX NSDI*, 2014, pp. 203–216.

[2] M. Dalton, D. Schultz, J. Adriaens, A. Arefin, A. Gupta, B. Fahs, D. Rubinstein, E. C. Zermeno, E. Rubow, J. A. Docauer *et al.*, "Andromeda: Performance, isolation, and velocity at scale in cloud network virtualization," in *USENIX NSDI*, 2018, pp. 373–387.

[3] D. Firestone, A. Putnam, S. Mundkur, D. Chiou, A. Dabagh, M. Andrewartha, H. Angepat, V. Bhanu, A. Caulfield, E. Chung *et al.*, "Azure accelerated networking: Smartnics in the public cloud," in *USENIX NSDI*, 2018, pp. 51–66.

[4] S. Fouladi, R. S. Wahby, B. Shacklett, K. Balasubramaniam, W. Zeng, R. Bhalerao, A. Sivaraman, G. Porter, and K. Winstein, "Encoding, fast and slow: Low-latency video processing using thousands of tiny threads." in *USENIX NSDI*, vol. 17, 2017, pp. 363–376.

[5] S. Fouladi, F. Romero, D. Iter, Q. Li, S. Chatterjee, C. Kozyrakis, M. Zaharia, and K. Winstein, "From laptop to lambda: Outsourcing everyday jobs to thousands of transient functional containers." in *USENIX ATC*, 2019, pp. 475–488.

[6] V. Sivaraman, S. Narayana, O. Rottenstreich, S. Muthukrishnan, and J. Rexford, "Heavy-hitter detection entirely in the data plane," in *ACM SOSR*, 2017, pp. 164–176.

[7] R. Ben-Basat, X. Chen, G. Einziger, and O. Rottenstreich, "Efficient measurement on programmable switches using probabilistic recirculation," in *IEEE ICNP*, 2018, pp. 313–323.

[8] Z. Liu, A. Manousis, G. Vorsanger, V. Sekar, and V. Braverman, "One sketch to rule them all: Rethinking network flow monitoring with univmon," in *ACM SIGCOMM*, 2016, pp. 101–114.

[9] P. Flajolet, É. Fusy, O. Gandouet, and F. Meunier, "Hyperloglog: the analysis of a near-optimal cardinality estimation algorithm," in *Discrete Mathematics and Theoretical Computer Science*. Discrete Mathematics and Theoretical Computer Science, 2007, pp. 137–156.

[10] H. Wang, C. Ma, S. Chen, and Y. Wang, "Fast and accurate cardinality estimation by self-morphing bitmaps," *IEEE/ACM Transactions on Networking*, vol. 30, no. 4, pp. 1674–1688, 2022.

[11] Z. Liu, R. Ben-Basat, G. Einziger, Y. Kassner, V. Braverman, R. Friedman, and V. Sekar, "Nitrosketch: Robust and general sketch-based monitoring in software switches," in *ACM SIGCOMM*, 2019, pp. 334–350.

[12] Q. Huang, X. Jin, P. P. Lee, R. Li, L. Tang, Y.-C. Chen, and G. Zhang, "Sketchvisor: Robust network measurement for software packet processing," in *ACM SIGCOMM*, 2017, pp. 113–126.

[13] T. Yang, J. Jiang, P. Liu, Q. Huang, J. Gong, Y. Zhou, R. Miao, X. Li, and S. Uhlig, "Elastic sketch: Adaptive and fast network-wide measurements," in *ACM SIGCOMM*, 2018, pp. 561–575.

[14] O. Alipourfard, M. Moshref, Y. Zhou, T. Yang, and M. Yu, "A comparison of performance and accuracy of measurement algorithms in software," in *ACM SOSR*, 2018, pp. 1–14.

[15] Y. Zhang, Z. Liu, R. Wang, T. Yang, J. Li, R. Miao, P. Liu, R. Zhang, and J. Jiang, "Cocosketch: high-performance sketch-based measurement over arbitrary partial key query," in *ACM SIGCOMM*, 2021, pp. 207–222.

[16] S. Miano, X. Chen, R. B. Basat, and G. Antichi, "Fast in-kernel traffic sketching in ebpf," *ACM SIGCOMM Computer Communication Review*, vol. 53, no. 1, 2023.

[17] OpenVSwitch, https://www.openvswitch.org/.

[18] D. Firestone, "Vfp: A virtual switch platform for host sdn in the public cloud." in *USENIX NSDI*, vol. 17, 2017, pp. 315–328.

[19] Vector Packet Processing, https://fd.io/technology/.

[20] M. Zhang, G. Li, S. Wang, C. Liu, A. Chen, H. Hu, G. Gu, Q. Li, M. Xu, and J. Wu, "Poseidon: Mitigating volumetric ddos attacks with programmable switches," in *NDSS*, 2020, pp. 823–835.

[21] Z. Liu, H. Namkung, G. Nikolaidis *et al.*, "Jaqen: A high-performance switch-native approach for detecting and mitigating volumetric ddos attacks with programmable switches," in *USENIX Security*, 2021.

[22] M. Moshref, M. Yu, R. Govindan, and A. Vahdat, "Dream: dynamic resource allocation for software-defined measurement," *ACM SIGCOMM Computer Communication Review*, vol. 44, no. 4, pp. 419–430, 2015.

[23] ——, "Scream: Sketch resource allocation for software-defined measurement," in *ACM CoNEXT*, 2015, p. 14.

[24] C. Misa, W. O'Connor, R. Durairajan, R. Rejaie, and W. Willinger, "Dynamic scheduling of approximate telemetry queries," in *USENIX NSDI*, 2022, pp. 701–717.

[25] A. Anup, L. Zaoxing, and S. Srinivasan, "Heterosketch: Coordinating network-wide monitoring in heterogeneous and dynamic networks," in *USENIX NSDI*, 2022, pp. 1–23.

[26] H. Zhu, T. Wang, Y. Hong, D. R. Ports, A. Sivaraman, and X. Jin, "Netvrm: Virtual register memory for programmable networks," in *USENIX NSDI*, 2022, pp. 155–170.

[27] eBPF, https://ebpf.io/.

[28] XDP, https://www.iovisor.org/technology/xdp.

[29] M. Xhonneux, F. Duchene, and O. Bonaventure, "Leveraging ebpf for programmable network functions with ipv6 segment routing," in *ACM CoNEXT*, 2018, pp. 67–72.

[30] M. A. Vieira, M. S. Castanho, R. D. Pacífico, E. R. Santos, E. P. C. Júnior, and L. F. Vieira, "Fast packet processing with ebpf and xdp: Concepts, code, challenges, and applications," *ACM Computing Surveys (CSUR)*, vol. 53, no. 1, pp. 1–36, 2020.

[31] M. Abranches, O. Michel, E. Keller, and S. Schmid, "Efficient network monitoring applications in the kernel with ebpf and xdp," in *IEEE NFV-SDN*, 2021, pp. 28–34.

[32] M. Yu, L. Jose, and R. Miao, "Software defined traffic measurement with opensketch," in *USENIX NSDI*, 2013, pp. 29–42.

[33] Y. Li, R. Miao, C. Kim, and M. Yu, "Flowradar: a better netflow for data centers," in *USENIX NSDI*, 2016, pp. 311–324.

[34] Q. Huang, S. Sheng, X. Chen *et al.*, "Toward nearly-zero-error sketching via compressive sensing," in *USENIX NSDI*, 2021, pp. 1027–1044.

[35] H. Namkung, Z. Liu, D. Kim *et al.*, "Sketchlib: Enabling efficient sketch-based monitoring on programmable switches," in *USENIX NSDI*, 2022, pp. 1–17.

[36] M. Charikar, K. Chen, and M. Farach-Colton, "Finding frequent items in data streams," *Theoretical Computer Science*, vol. 312, no. 1, pp. 3–15, 2004.

[37] H. Zheng, C. Tian, T. Yang, H. Lin, C. Liu, Z. Zhang, W. Dou, and G. Chen, "Flymon: enabling on-the-fly task reconfiguration for network measurement," in *ACM SIGCOMM*, 2022, pp. 486–502.

[38] Y. Zhou, Y. Zhang, C. Ma, S. Chen, and O. O. Odegbile, "Generalized sketch families for network traffic measurement," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 3, no. 3, pp. 1–34, 2019.

[39] Perf, http://perf.wiki.kernel.org.

[40] NetFilter, https://www.netfilter.org/.

[41] bcc, https://github.com/iovisor/bcc.

[42] G. Cormode and S. Muthukrishnan, "An improved data stream summary: the count-min sketch and its applications," *Journal of Algorithms*, vol. 55, no. 1, pp. 58–75, 2005.

[43] Q. Huang, P. P. Lee, and Y. Bao, "Sketchlearn: relieving user burdens in approximate measurement with automated statistical inference," in *ACM SIGCOMM*, 2018, pp. 576–590.

[44] PktGen, https://pktgen-dpdk.readthedocs.io/.

[45] The CAIDA Anonymized Internet Traces, http://www.caida.org/data/overview/.

[46] X. Chen, Q. Xiao, H. Liu, Q. Huang, D. Zhang, X. Liu, L. Hu, H. Zhou, C. Wu, and K. Ren, "Eagle: Toward scalable and near-optimal network-wide sketch deployment in network

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:50 UTC from IEEE Xplore. Restrictions apply.

measurement," in *ACM SIGCOMM*, 2024, pp. 291–310.
[47] G. Cormode and S. Muthukrishnan, "An improved data stream summary: The count-min sketch and its applications," *Journal of Algorithms*, vol. 55, no. 1, pp. 58–75, 2004.
[48] M. C. A, K. C. B, and F. C. C, "Finding frequent items in data streams," *Theoretical Computer Science*, vol. 312, no. 1, pp. 3–15, 2002.
[49] R. Ben Basat, G. Einziger, R. Friedman, M. C. Luizelli, and E. Waisbard, "Constant time updates in hierarchical heavy hitters," in *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, ser. SIGCOMM '17. New York, NY, USA: Association for Computing Machinery, 2017, p. 127–140. [Online]. Available: https://doi.org/10.1145/3098822.3098832
[50] G. Cormode, F. Korn, S. Muthukrishnan, and D. Srivastava, "Finding hierarchical heavy hitters in streaming data," *ACM Trans. Knowl. Discov. Data*, vol. 1, no. 4, Feb. 2008. [Online]. Available: https://doi.org/10.1145/1324172.1324174
[51] P. Flajolet and G. N. Martin, "Probabilistic counting algorithms for data base applications," *J. Comput. Syst. Sci.*, vol. 31, no. 2, p. 182–209, Sep. 1985. [Online]. Available: https://doi.org/10.1016/0022-0000(85)90041-8
[52] C. Estan, G. Varghese, and M. Fisk, "Bitmap algorithms for counting active flows on high speed links," in *Proceedings of the 3rd ACM SIGCOMM Conference on Internet Measurement*, ser. IMC '03. New York, NY, USA: Association for Computing Machinery, 2003, p. 153–166. [Online]. Available: https://doi.org/10.1145/948205.948225
[53] M. Durand and P. Flajolet, "Loglog counting of large cardinalities," in *European Symposium on Algorithms*. Springer, 2003, pp. 605–617.
[54] A. Lall, V. Sekar, M. Ogihara, J. Xu, and H. Zhang, "Data streaming algorithms for estimating entropy of network traffic," in *Proceedings of the Joint International Conference on Measurement and Modeling of Computer Systems*, ser. SIGMETRICS '06/Performance '06. New York, NY, USA: Association for Computing Machinery, 2006, p. 145–156. [Online]. Available: https://doi.org/10.1145/1140277.1140295
[55] B. Krishnamurthy, S. Sen, Y. Zhang, and Y. Chen, "Sketch-based change detection: methods, evaluation, and applications," in *Proceedings of the 3rd ACM SIGCOMM Conference on Internet Measurement*, ser. IMC '03. New York, NY, USA: Association for Computing Machinery, 2003, p. 234–247. [Online]. Available: https://doi.org/10.1145/948205.948236
[56] C. H. Song, P. G. Kannan, B. K. H. Low, and M. C. Chan, "Fcm-sketch: generic network measurements with data plane support," in *ACM CoNEXT*, 2020, pp. 78–92.
[57] Z. Liu, A. Manousis, G. Vorsanger, V. Sekar, and V. Braverman, "One sketch to rule them all: Rethinking network flow monitoring with univmon," in *Proceedings of the SIGCOMM '16*. Association for Computing Machinery, 2016, p. 101–114.
[58] Barefoot Tofino, https://www.barefootnetworks.com/technology/#tofino.
[59] X. Chen, Q. Huang, P. Wang, H. Liu, Y. Chen, D. Zhang, H. Zhou, and C. Wu, "Mtp: Avoiding control plane overload with measurement task placement," in *IEEE INFOCOM*, 2021, pp. 1–10.
[60] A. Roy, H. Zeng, J. Bagga, G. Porter, and A. C. Snoeren, "Inside the social network's (datacenter) network," in *ACM SIGCOMM*, 2015, pp. 123–137.
[61] MAWI, https://mawi.wide.ad.jp/mawi/.
[62] IMC, https://pages.cs.wisc.edu/~tbenson/IMC10_Data.html.
[63] X. Jin, X. Li, H. Zhang *et al.*, "Netcache: Balancing key-value stores with fast in-network caching," in *ACM SOSP*, 2017, pp. 121–136.
[64] B. Yang, D. Shen, J. Zhang, H. Yang, L. Zhao, B. Wang, G. Liu, and K. Chen, "enetstl: Towards an in-kernel library for high-performance ebpf-based network functions," in *ACM EuroSys*, 2025, pp. 42–58.

Authorized licensed use limited to: SICHUAN UNIVERSITY. Downloaded on September 18,2026 at 09:30:50 UTC from IEEE Xplore. Restrictions apply.